

INFORME DE DESARROLLO

Sistema de estacionamiento

1. Objetivo

El objetivo fue desarrollar un programa de consola capaz de controlar un estacionamiento de 64 espacios. El sistema permite ingresar, retirar y buscar vehículos, mostrar el tablero, calcular la ruta más corta entre la entrada y la salida, y llevar el control de los ingresos.

2. Resumen del desarrollo

El programa fue desarrollado en Java con NetBeans y Maven. Para almacenar la información se utilizaron dos matrices de tipo String de 10 x 10: una representa el estado del tablero y otra guarda las placas. La lógica se dividió en las clases Estacionamiento, Menu, Tablero y Validaciones. No se utilizaron colecciones dinámicas.

3. Problemas encontrados y soluciones adoptadas

3.1 El vehículo se registraba con un pago insuficiente

Problema: Al principio, el ciclo del pago solamente se repetía cuando el monto era negativo. Por esa razón, un pago como Q7 permitía continuar.

Solución: Se cambió la condición del ciclo a $\text{monto} < 10$. Ahora el programa vuelve a solicitar el pago hasta recibir Q10 o más.

3.2 Se cobraba antes de comprobar el espacio

Problema: El pago se realizaba antes de verificar si la posición estaba libre, por lo que podía cobrarse aunque el espacio ya estuviera ocupado.

Solución: La llamada a `realizarPago()` se movió dentro de la condición que comprueba que la celda contiene la letra L.

3.3 El menú fallaba al ingresar letras

Problema: El método `nextInt()` produce un error cuando el usuario escribe una letra en lugar de un número.

Solución: Se utilizó `hasNextInt()` antes de leer la opción. Si la entrada no es numérica, se muestra un mensaje y se descarta el dato incorrecto.

3.4 Formato incorrecto o placa repetida

Problema: Era necesario evitar placas incompletas, con p minúscula, números o letras en posiciones incorrectas, y placas ya registradas.

Solución: Se creó la clase Validaciones. El formato P####LLL se revisa con métodos de String y Character, y la matriz de placas se recorre para detectar duplicados.

3.5 Reutilización de la búsqueda al retirar

Problema: Buscar y retirar un vehículo necesitaban localizar la misma placa, lo cual podía duplicar código.

Solución: El método buscarVehiculo() devuelve un arreglo de dos posiciones con la fila y la columna. Retirar reutiliza ese resultado y libera ambas matrices.

3.6 Prueba del estacionamiento lleno

Problema: Ingresar manualmente 64 vehículos hacía muy lenta la prueba de capacidad máxima.

Solución: Se creó temporalmente un método de prueba que llenaba las 64 posiciones con A. Después de comprobar el mensaje de estacionamiento lleno, el método quedó comentado.

3.7 El archivo JAR no encontraba la clase principal

Problema: Al ejecutar el JAR aparecía el mensaje de que no se encontraba el atributo de manifiesto principal.

Solución: Se agregó ipc1.practica1.Estacionamiento como mainClass en la configuración de maven-jar-plugin y se volvió a compilar el proyecto.

4. Evidencias del funcionamiento

Se realizaron pruebas desde la consola de NetBeans para comprobar las funciones principales. Los resultados obtenidos se resumen a continuación.

Prueba	Dato utilizado	Resultado observado
Ingreso correcto	P123ABC, fila 1, columna 1 y Q10	El vehículo se registra y la posición cambia de L a A.
Pago insuficiente	Q7	Se informa que el monto es insuficiente y se solicita nuevamente.
Placa inválida	p123abc	La placa es rechazada porque no cumple el formato P####LLL.
Placa repetida	P123ABC dos veces	El segundo ingreso es rechazado.
Búsqueda	P123ABC	Se muestran la fila y la columna del vehículo.
Retiro	P123ABC	La posición vuelve a L y la placa se elimina de la matriz.
Tablero	Opción 3	Se muestra la matriz y el total de espacios libres y ocupados.
Ruta exterior	Opción 5	Se comparan ambas rutas y se recomienda la menor.

Prueba	Dato utilizado	Resultado observado
Parqueo lleno	64 posiciones con A	El sistema informa que no hay espacio disponible.

5. Decisiones de implementación

Se trabajó con matrices porque el tamaño del estacionamiento es fijo. La clase Tablero concentra las operaciones que modifican los espacios, mientras que Menu controla la interacción y Validaciones revisa las placas. Esta organización permitió mantener el programa sencillo y evitar repetir código.

6. Conclusión

El programa cumple con las funciones solicitadas y maneja los errores principales que puede cometer el usuario. Las pruebas permitieron corregir problemas de pago, ingreso de datos y disponibilidad de espacios. Como resultado, se obtuvo una aplicación de consola sencilla, ordenada y funcional.